tensor4all development update

Contributors, tree methods, tenferro, and the kernel libraries

tensor4all developers

tensor4all meeting, 2026-10-27

Who contributed since September 22

Name	Where
Samuel Badr	tensor4all-rs
Simon Becker	tensor4all-rs: tree network storage (HDF5)
Lingrui Cheng	tensor4all-rs: tree patching, TreeRSI, tree reliability
Lukas Devos	tcontract-rs: the original contraction kernels
Ken Inayoshi	tensor4all-rs: performance, quantics transform precision
Martin Mikkelsen	tlinalg-rs, t4a-faer, tensor4all-rs, tenferro-rs
Marc K. Ritter	AlternatingCrossInterpolation.jl
Rihito Sakurai	tensor4all-rs: TCI pivot search fixes
Hiroshi Shinaoka	most repositories
Satoshi Terasaki	tenferro-rs, tenferro-benchmark, CI bot
Ryo Watanabe	t4a-faer

Commit authors and co-authors on the default branches of tensor4all repositories, 2026-09-22 to 2026-10-27, alphabetical. Also upstream faer authors (via sync) and AI coding agents under human review.

Activity since September 22

11
people committed

33
repositories touched

15
new public repositories

tensor4all-rs: tree methods

tenferro-rs: execution engine, CUDA backend

cpueinsum, tcontract, tlinalg: split out of tenferro

New repositories include tcontract-rs, cpueinsum-rs, tlinalg-rs, the faer fork, t4a-cuda-kernels, t4a-gpu-backend, threadteam-rs and their benchmark repositories.

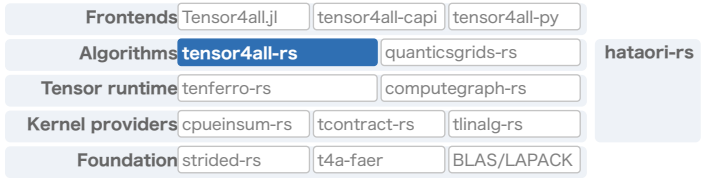
The stack

Frontends Julia, C, Python	**Tensor4all.jl** Julia frontend, best effort	**tensor4all-capi** C API over tensor4all-rs	**tensor4all-py** PyO3 prototype
Algorithms TT, TCI, tree networks	**tensor4all-rs** TT and TCI stack, network stack	**quanticsgrids-rs** quantics grids, coordinates	
Tensor runtime tensors, AD, backends	**tenferro-rs** tensor types, autodiff, CPU/GPU	**computegraph-rs** graph IR for traced AD	
Kernel providers contraction, linalg	**cpueinsum-rs** N-ary einsum, given order	**tcontract-rs** binary contraction, C ABI	**tlinalg-rs** batched SVD, QR, eigh, LU
Foundation views, dense kernels	**strided-rs** shared strided view types	**t4a-faer** our fork of faer	**BLAS/LAPACK** vendor library, optional

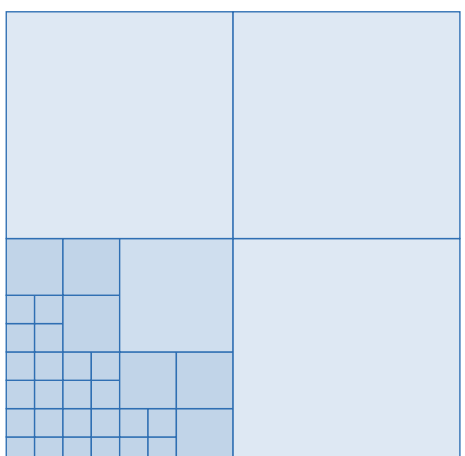
hataori-rs
map and pmap over threads and MPI

Interactive version: tensor4all.org/tensor4all-meta/stack/

Adaptive patching on trees



Published for tensor trains; **Lingrui Cheng extended it to trees.**

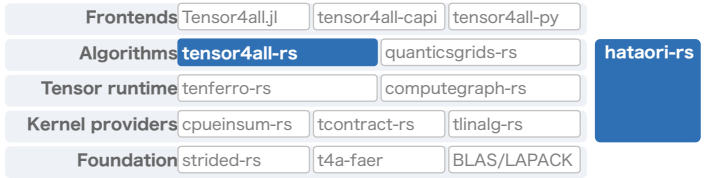


- Split the domain **only where the function is hard**
- Every patch is a small tree with the same shape
- Works for chains and branched trees alike

On one test the patched run was slower and less accurate than a single network. When patching pays off is still open.

G. Grosso, M. K. Ritter, S. Rohshap, S. Badr, A. Kauch, M. Wallerberger, J. von Delft, H. Shinaoka, Adaptive patching for tensor train computations, arXiv:2602.22372. Trees: tensor4all-rs PR 900

Parallel patching with hataori



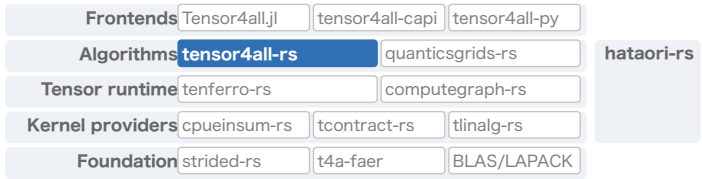
one big problem → many independent patches → hataori: threads or MPI ranks → one result

- Patches are independent, so they can run on different cores or nodes
- **hataori-rs** hands out the work; each worker uses its own tenferro engine
- Chains and trees share one implementation; the chain-only crate was removed

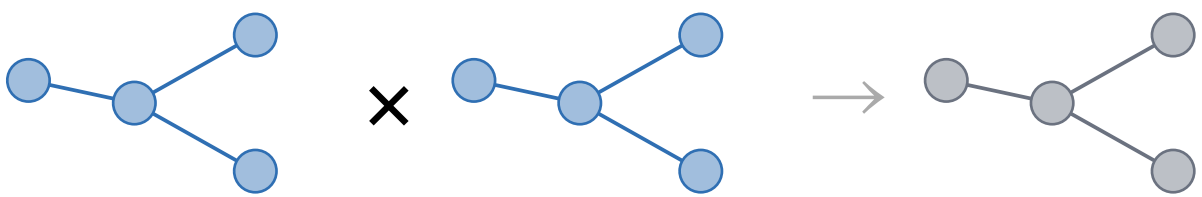
Tested for correctness on 1 and 2 MPI ranks. Scaling not measured yet.

tensor4all-rs PRs 1005, 1009 hataori-rs PRs 25, 27

TreeRSI



Published for tensor trains; **Lingrui Cheng extended it to trees.**

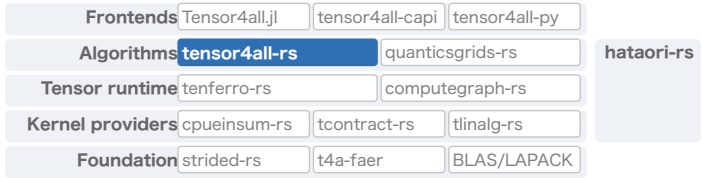


tree A × tree B → A·B, already small

- Element-by-element product of two trees, compressed in one pass
- Checked against the original authors' code, with the same random numbers

Z. Meng, Y. Khoo, J. Li, E. M. Stoudenmire, Recursive sketched interpolation: efficient Hadamard products of tensor trains, arXiv:2602.17974.

Other fixes in tensor4all-rs



Convergence reports

Lingrui Cheng

Says “rank limit reached” instead of a false “converged”.

TCI pivots

Rihito Sakurai

Pivot search no longer stops at zero residuals or forgets good moves.

Speed

Ken Inayoshi, Martin Mikkelsen

Fewer copies in core loops; column-major grids in TCI.

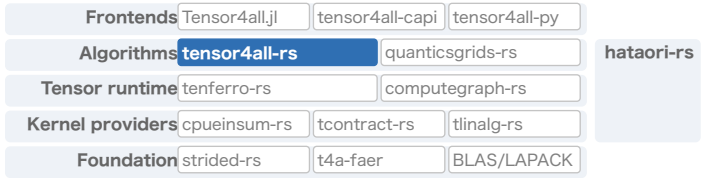
Storage

Simon Becker

Tree networks saved to HDF5 keep their canonical form.

Full details of the tree work: slides/tensor4all20261027-lingrui-cheng.pdf

Remaining work



Patched products

biggest gap

Multiply two patched operators (MPO-MPO) end to end.

Error checks

cheaper and safer

Catch local errors that sampling misses, at lower cost; fix remaining TreeACI convergence issues.

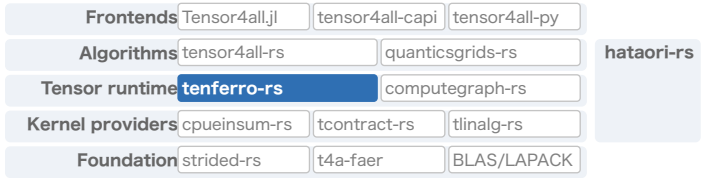
Scaling

not measured yet

Run the MPI patching at scale and measure.

Source: Lingrui Cheng's slides, "Remaining work".

tenferro execution engine



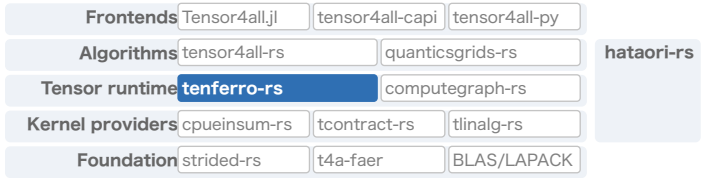
your program → creates an **engine**: threads, memory, GPU stream → prepare once → run many times

- The engine is an ordinary value: you create it and pass it to each call
- Two engines can run side by side, for example one per MPI rank or per patch
- Small operations stay on the calling thread

This is a breaking change; the old session API was removed.

tenferro-rs 2206, PR 2209

New CUDA backend



tenferro GPU engine

↓

t4a-cuda-kernels
our own CUDA kernels, C ABI

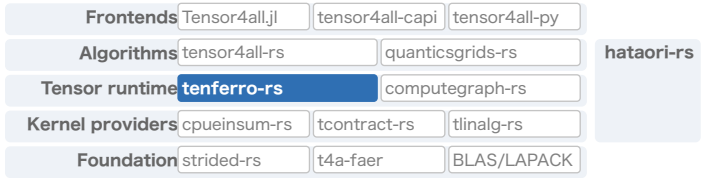
cuBLAS, cuTENSOR
vendor libraries

t4a-gpu-backend: streams, memory, launches

- Replaces the earlier CubeCL-based CUDA path
- The kernels use no tenferro types, so other CUDA codes can call them
- Matrix products via cuBLAS, contractions via cuTENSOR
- Migration is in progress, piece by piece; tested on an A100

tenferro-rs 2208, PRs 2250 to 2263 t4a-cuda-kernels; t4a-gpu-backend.

tenferro-benchmark

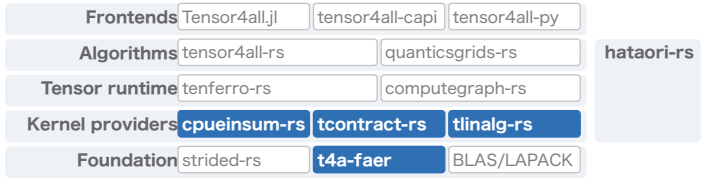


tenferro main → CPU: every night, Linux and macOS GPU: A100, every weekend → PyTorch, JAX, Julia, HPTT, cuTENSOR → dashboard

- Runs start automatically from GitHub Actions; the GPU runs use rented A100s
- Raw data and settings for every run are kept in the repository
- Example, A100, a tensor network contraction (f32): tenferro 49 ms, PyTorch 111 ms
- Dashboard: tensor4all.org/tenferro-benchmark

github.com/tensor4all/tenferro-benchmark; example from result/nvidia-gpu/gpu/tensornetwork.md, 2026-10-24, tenferro 06ac247. Most changes since September 22 by Satoshi Terasaki.

Why split out the kernels



Rust: tenferro

Julia

Python

C, Fortran codes

→

C API

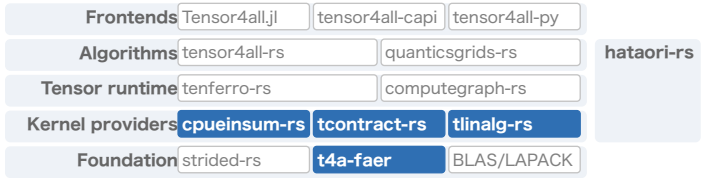
Engine

tcontract · cpueinsum · tlinalg · t4a-faer

Started with **Lukas Devos** (CCQ, developer of TensorKit.jl) and his Rust prototype.

- **Julia:** call precompiled kernels, less precompile time (future)
- Fast kernels are too much work for one group to maintain
- **We want to maintain them together with other groups**
- Planned: CPU kernel optimization with Guo Chen's group at HKUST(GZ)

Four libraries



tcontract-rs

multiply two tensors

From **Lukas Devos's prototype**. C API follows **TAPP**.

cpueinsum-rs

multiply many tensors

Cut out of tenferro. Lower per-step overhead.

tlinalg-rs

SVD, QR, LU, eigen

Thin layer over t4a-faer. **Batch** API like PyTorch.

t4a-faer

fork of faer

New parallel model, bug fixes, **AI-driven** optimization.

All four now on crates.io and used by tenferro.

tcontract-rs



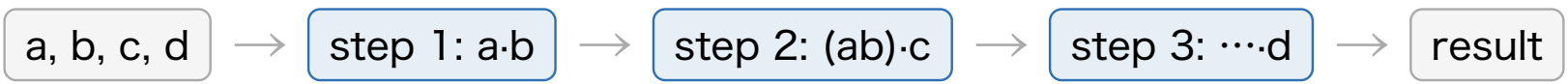
A, B with any strides → packed straight into small tiles → fast inner kernel (AVX, NEON) → D = A·B + C

- No transpose or copy of the inputs first
- f32, f64 and complex; batch indices, traces, conjugation
- C API follows **TAPP**, a community standard for tensor products
- One shared library, `libtcontract_capi`, for any language with a C FFI

TAPP: arXiv:2601.07827. tcontract-rs README.

cpueinsum-rs





- **You** choose the order; cpueinsum plans every step once
- One reusable scratch buffer for all the intermediates
- Not included: order search, `"ij,jk->ik"` strings, GPUs
- Usable without tenferro

tlinalg-rs: batches



Like PyTorch, tlinalg takes a **batch**: many matrices of the same shape in one call.

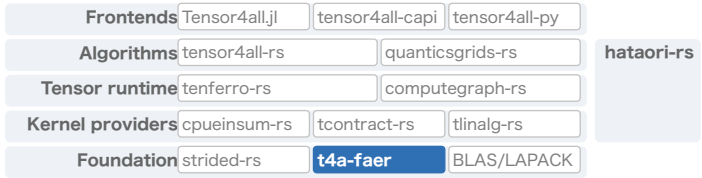
With 4 threads there are two ways to run it:

1. Give each thread its own share of the batch, a contiguous block of matrices.
2. Factorize the matrices one by one, each with all 4 threads.

For small matrices the threads in the second way spend most of their time coordinating, so **the first way is faster**. By default tlinalg picks the way itself; a caller can also fix it.

tlinalg-rs 0.6. The default policy estimates each matrix's work and splits the batch across threads only when that repays the cost of starting them.

t4a-faer



Fork of faer

dense linear algebra in Rust

We use it for fast experiments, mostly run by AI agents.

What we changed

Parallel model, bug fixes, faster QR and LU paths.

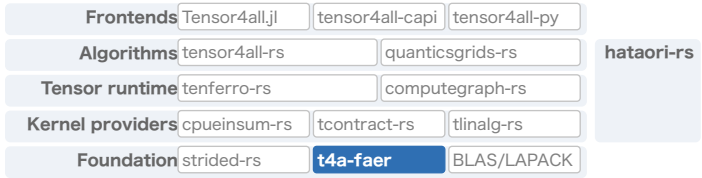
Experiment log

PATCHSET.md

Each change is recorded as kept or dropped, with the reason.

faer by Sarah Quiñones. Our fork: github.com/tensor4all/faer, crates.io t4a-faer.

faer-benchmark

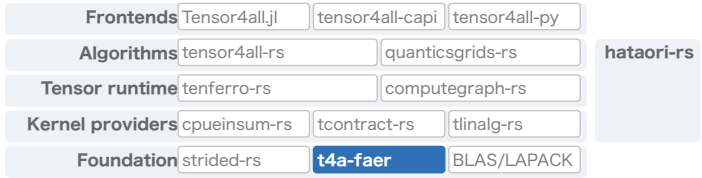


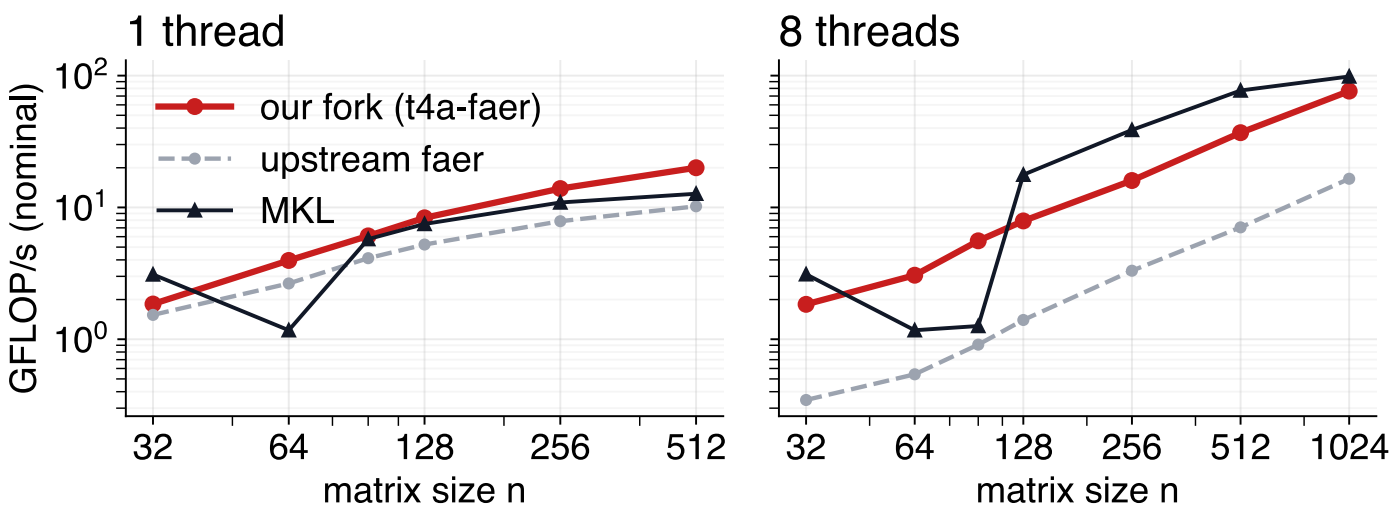
every commit of the fork → same machine, same protocol → fork vs upstream faer vs MKL, OpenBLAS → public pages

- Nine families: GEMM, LU, Cholesky, QR, SVD, eigenvalues, …
- Each result is tied to a commit and a machine; results from older commits are marked
- An unchanged control row checks the machine; if it moves, the result is not accepted
- Raw data, scripts and every run's settings are in the repository

github.com/tensor4all/faer-benchmark. Sibling repositories: tlinalg-benchmark, tcontract-benchmark, cpueinsum-benchmark, tenferro-benchmark.

QR, 1 vs 8 threads





1 thread: our fork beats upstream faer and MKL.

8 threads: MKL still leads. **Parallel scaling is the next target.**

faer-benchmark, AMD Zen 5, f64, nominal GFLOP/s ($4n^3/3$); fork 50c2b98 (n = 1024: a5e6b7d) vs upstream faer 0.24.4. QR with an explicit Q shows the same trend. Provisional: development host.

Regular benchmarking

AI agents write most of this code.

Public, repeatable benchmarks let others check the results.

- tenferro-benchmark already runs on a schedule: CPU every night, A100 every weekend
- faer-benchmark runs on a development host for now; a dedicated machine will rerun it regularly
- We will show the trends at these meetings

Discussion

- **tensor4all-rs:** which tree workloads should we test with real users?
- **tenferro:** who needs the CUDA backend first?
- **Kernels:** who else wants to share them with us?
- **t4a-faer:** send our fixes back to upstream faer?

References

- Tree methods in detail (Lingrui Cheng): `slides/tensor4all20261027-lingrui-cheng.pdf` and its notes
- Adaptive patching: G. Grosso et al., arXiv:2602.22372
- RSI: Z. Meng, Y. Khoo, J. Li, E. M. Stoudenmire, arXiv:2602.17974
- TAPP: arXiv:2601.07827
- Stack map: tensor4all.org/tensor4all-meta/stack/
- Code: github.com/tensor4all (tensor4all-rs, tenferro-rs, tcontract-rs, cpueinsum-rs, tlinalg-rs, faer)